Introduction to Algorithms

Personal Notes

Mihir Rao

Contents

1 Introduction

An algorithms course is fundamentally about three tasks: state a computational problem precisely, design a procedure that solves every valid instance, and communicate why the procedure is both correct and efficient. The distinction between these tasks matters. A problem describes what outputs are acceptable; an algorithm chooses how to produce one; a proof and a running-time analysis justify the choice.

1.1 Problems and Algorithms

Definition 1.1 (Computational problem)**.** A *computational problem* is a binary relation between a collection of valid inputs and their acceptable outputs. Equivalently, one may specify a predicate that can be checked to decide whether a proposed output is correct for a given input.

This relational view allows an input to have several correct outputs. For example, if a collection contains several pairs of people with a shared birthday, returning any one such pair is acceptable. Algorithms are expected to work over a general, arbitrarily large input space rather than for one fixed instance.

Definition 1.2 (Algorithm)**.** An *algorithm* is a finite procedure that maps each valid input to an output. It *solves* a problem when its output is acceptable for every valid input.

Consider the problem of finding two students with the same birthday. A simple algorithm processes the students in order. Before processing position k, it keeps the first k students in a record. It compares the next birthday with every entry already stored, returns a matching pair if one exists, and otherwise appends the new student. If the scan ends, no pair exists.

```
def find_shared_birthday(students):
    """Return two names sharing a birthday, or None."""
    n = len(students)
    record = StaticArray(n)

    for k in range(n):
        current_name, current_day = students[k]
        for i in range(k):
            previous_name, previous_day = record.get_at(i)
            if current_day == previous_day:
                return current_name, previous_name
        record.set_at(k, students[k])

    return None
```

1.2 Correctness

Testing a few inputs cannot establish correctness on an unbounded input space. A loop or recursive call instead suggests an inductive proof whose statement tracks the portion of the input already processed.

Theorem 1.3. *The birthday-matching algorithm returns a pair if and only if two students in the input share a birthday.*

Proof. First suppose the algorithm returns two names. It does so only after directly comparing their stored birthdays and finding them equal, so every returned pair is valid.
For the converse, induct on the number k of processed students. As the induction hypothesis, assume that if the first k students contain a match, the algorithm has already returned one. The statement is vacuous at $k = 0$. Now process student $k + 1$. If a match lies among the first k students, the induction hypothesis applies. Otherwise, any match among the first $k + 1$ students must involve student $k + 1$. The inner loop compares that student's birthday with each of the first k birthdays and therefore finds the match. Thus the claim holds for every prefix, including the complete input. □

The proof separates two obligations that recur throughout algorithm design: *soundness*, meaning every produced answer is valid, and *completeness*, meaning an answer is found whenever one exists.

1.3 Efficiency and Asymptotic Growth

Wall-clock time depends on hardware, language, compiler, and system load. To compare algorithms independently of those details, analysis counts elementary operations in a stated model of computation and expresses that count as a function of input size, conventionally n.

Definition 1.4 (Asymptotic bounds)**.** For eventually nonnegative functions f and g:

- $f(n) = O(g(n))$ if constants $c > 0$ and n_0 exist such that $f(n) \le c\,g(n)$ for all $n \ge n_0$;
- $f(n) = \Omega(g(n))$ if constants $c > 0$ and n_0 exist such that $f(n) \ge c\,g(n)$ for all $n \ge n_0$;
- $f(n) = \Theta(g(n))$ when both bounds hold.

Asymptotic notation suppresses constant factors and lower-order terms so that the dominant growth rate remains visible. For large inputs, the progression

$$\Theta(1), \quad \Theta(\log n), \quad \Theta(n), \quad \Theta(n \log n), \quad \Theta(n^2), \quad 2^{\Theta(n)}$$

represents increasingly severe scaling. Polynomial-time algorithms have running time $O(n^c)$

for some constant c and are the course's baseline notion of efficiency, although a lower-degree polynomial is often far more useful in practice.

For birthday matching, allocating the static array costs $\Theta(n)$. On outer iteration k, the inner loop performs at most k constant-time comparisons. The worst-case running time is therefore

$$\begin{aligned} T(n) &= \Theta(n) + \sum_{k=0}^{n-1} \Theta(k+1) \\ &= \Theta(n) + \Theta\left(\frac{n(n+1)}{2}\right) \\ &= \Theta(n^2). \end{aligned}$$

The algorithm is polynomial, but the quadratic scan signals that a better record structure may improve it.

1.4 The Word-RAM Model

An operation count has meaning only after fixing which operations count as constant time. The course uses the *word random-access machine*, or Word-RAM.

Definition 1.5 (Word-RAM)**.** A w-bit Word-RAM consists of an addressable memory of w-bit words and a processor that performs arithmetic, comparisons, Boolean and bitwise operations, and reads or writes a constant number of words in $O(1)$ time.

A word must be large enough to encode any relevant memory address. If the algorithm may address n words, this requires

$$w \geq \lceil \log_2 n \rceil.$$

At the same time, treating an arbitrarily large integer as one constant-time word would hide real work. The word-size assumption is what connects the abstract machine to realistic finite hardware. Higher-level languages such as Python provide richer operations, but those operations are ultimately implemented by sequences of Word-RAM steps and may not themselves be constant time.

1.5 Data Structures and Interfaces

Definition 1.6 (Data structure)**.** A *data structure* stores a nonconstant amount of data and supports a specified collection of operations. That collection of operations is its *interface*; the representation and algorithms realizing the interface form its implementation.

Separating interface from implementation lets two structures support the same operations with different performance guarantees. A *sequence* interface organizes items by extrinsic position, such as first, last, or ith. A *set* interface organizes items by intrinsic keys and supports queries based on those keys.

The static array is the first concrete example. It stores a fixed number of fixed-width slots in contiguous, addressable memory:

Operation	Meaning	Running time
`StaticArray(n)`	Allocate and initialize n slots	$\Theta(n)$
`get_at(i)`	Read the word at index i	$\Theta(1)$
`set_at(i, x)`	Write word x at index i	$\Theta(1)$

A slot can contain either a value or the address of a larger object. Fixed length makes random access simple but prevents the array from growing; dynamic arrays address that limitation in the next lecture.

1.6 A General Design Strategy

When facing a new algorithms problem, first try to reduce it to a familiar problem whose data structure or algorithm already provides the needed operation. The birthday example asks repeatedly whether a key has appeared, so its quadratic static-array record should eventually be replaced by a data structure supporting faster membership queries.

If no known reduction is available, design a procedure using a standard paradigm: brute force, decrease-and-conquer, divide-and-conquer, dynamic programming, or a greedy incremental construction. In either case the same three questions close the loop:

1. What precisely constitutes a correct output?
2. Why does the algorithm always produce one?
3. How many operations does it require in the chosen model?

Source: MIT OpenCourseWare, MIT 6.006 Introduction to Algorithms, Spring 2020, Lecture 1: Introduction, by Erik Demaine, Jason Ku, and Justin Solomon. Official lecture resource.